EXPERIMENT 14

Pro-Code Extension of a Zoho Creator Application

Vehicle Service Request Management System — Priority Scoring with Deluge and Zoho Catalyst

Submitted by	Suhritha Sri
Department	Computer Science and Business Systems (CSBS)
Register number	241401110
Application	Vehicle Service Request Management System
Platform	Zoho Creator (low-code) with Zoho Catalyst (pro-code)

1. Aim

To study how a Vehicle Service Request Management System built in Zoho Creator can be extended beyond standard workflow actions by combining a reusable Deluge custom function with a serverless Zoho Catalyst function. The worked example assigns a service-priority level to a submitted vehicle request using details such as vehicle age, service type, breakdown status and urgency. Creator sends the request data to the external function and uses the returned priority to update the service record. The code and workflow in this report are illustrative and are not presented as a deployed implementation.

2. Procedure

Step	Activity
1. Identify the need	Choose a requirement beyond basic actions: prioritising urgent repairs and breakdowns.
2. Compare options	Compare workflow actions, Deluge functions, Creator Custom APIs and Catalyst.
3. Design the architecture	Connect the Creator workflow to a Catalyst Advanced I/O function using a managed connection.
4. Write the Creator function	Send request fields as JSON with invokeurl and define a safe default.
5. Write the Catalyst function	Validate the payload, score the request and return JSON.
6. Plan execution flow	Show the success path and the error path without losing the request.
7. Record precautions	Review validation, access control, timeouts, maintenance and cost.

Note: The endpoint URL and connection name are placeholders. The example code has not been deployed or executed. Priority rules are demonstration rules and should be approved by the service centre before real use.

3. Output

3.1 Extension options compared

Option	Description	Skill level	Typical use
Workflow actions	Built-in actions such as notifications, field updates and status changes.	Low	Routine request processing.
Deluge custom function	Reusable function called from a Creator workflow.	Low to medium	Shared service-priority rules.
Creator Custom API	REST endpoint exposed by a Creator application.	Medium	External systems submitting or retrieving service requests.
Catalyst function	Serverless Node.js, Java or Python function invoked over HTTPS.	Medium to high	Complex rules, external libraries or integrations.

3.2 Architecture of the extension

ZOHO CREATOR Vehicle service request form	WORKFLOW On successful submission	ZOHO CATALYST Advanced I/O function
Vehicle, service type, urgency and notes	Deluge custom function packages JSON and calls API	Validates input and applies priority-scoring rules
Record updated with priority and status	HTTPS request / JSON response	Returns Normal, Medium or High priority

Fig. 1: Creator manages the form and record; Catalyst performs the example scoring logic and returns JSON.

When a service request is submitted, a Creator workflow calls the Deluge function. The function sends the selected request fields through a managed HTTPS connection to Catalyst. Catalyst validates the values, applies the example rules and returns a JSON priority. Creator can then update the request record.

Vehicle service dashboard screenshot

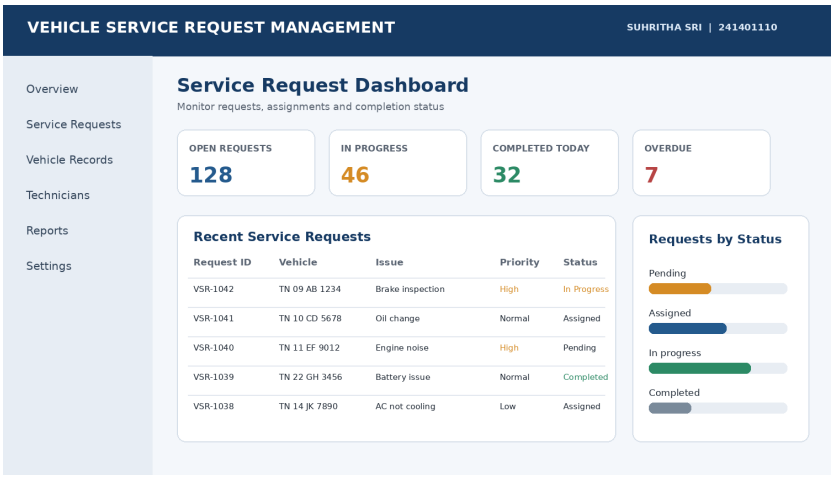


Fig. 2: Dashboard showing request totals and current service-request status.

3.3 Deluge custom function (illustrative)

```
// Reusable function: calculateServicePriority
string calculateServicePriority(string serviceType, string urgency, string breakdown)
{
    priority = "Normal"; // safe default
    try
    {
        payload = Map();
        payload.put("service_type", serviceType);
        payload.put("urgency", urgency);
        payload.put("breakdown", breakdown);
        response = invokeurl
        [
            url : "<Catalyst Advanced I/O endpoint>/priority"
            type : POST
            parameters : payload.toString()
            headers : {"Content-Type":"application/json"}
            connection : "catalyst_connection"
        ];
        if(response.get("priority") != null)
        {
            priority = response.get("priority");
        }
    }
    catch (e)
    {
        info "Service-priority call failed: " + e;
    }
    return priority;
}

// Example workflow call after request submission
priority = thisapp.calculateServicePriority(
    input.service_type.toString(),
    input.urgency.toString(),
    input.breakdown_status.toString()
);
Vehicle_Service_Requests[ID == input.ID].priority = priority;
```

The Creator function packages the selected service fields and calls the external endpoint. If the request fails or no priority is returned, it retains the safe default. Endpoint and connection names are placeholders; the authentication method, response shape and failure behaviour must be confirmed and tested before deployment.

3.4 Catalyst Advanced I/O function in Node.js (illustrative)

```
const express = require("express");
const app = express();
app.use(express.json());

app.post("/priority", (req, res) => {
  const { service_type, urgency, breakdown } = req.body || {};

  if (typeof service_type !== "string" ||
      typeof urgency !== "string" ||
      typeof breakdown !== "string") {
    return res.status(400).json({ error: "Invalid request fields" });
  }

  let score = 0;
  const type = service_type.toLowerCase();
  const level = urgency.toLowerCase();
  const isBreakdown = breakdown.toLowerCase();

  if (level === "high" || level === "urgent") score += 3;
  if (isBreakdown === "yes" || isBreakdown === "true") score += 3;
  if (type.includes("engine") || type.includes("brake")) score += 2;

  const priority = score >= 5 ? "High" :
                   score >= 2 ? "Medium" : "Normal";
  return res.status(200).json({ priority });
});

module.exports = app;
```

The example validates expected fields, scores urgency, breakdown status and selected repair categories, then returns High, Medium or Normal. These thresholds are illustrative only; a real service centre should define and test its own policy.

3.6 Execution flow

START Vehicle service request submitted
Creator workflow calls Deluge function
Function creates JSON payload and sends HTTPS request
Catalyst validates fields and calculates example score
Valid response received?
YES — Update request priority using returned value
NO — Log error and retain default priority; keep request record
END

Fig. 5: Execution flow from service request submission to priority update, including the error path.

A failed external request must not cause the service request to be lost. The record should remain saved with a default priority while the failure is logged for review. A production implementation should include timeouts, response validation, retry policy and clear staff-facing error handling.

3.7 Choosing an extension type

Question	Recommended direction
Can a built-in action handle the requirement?	Prefer a standard Creator workflow action for simple notifications and field updates.
Is the rule reusable inside Creator?	Use a Deluge custom function.
Must an external system call the Creator application?	Consider a Creator Custom API.
Are advanced libraries, complex processing or outside services needed?	Consider a Catalyst function and review security and operations.
No option fits well?	Reconsider and simplify the requirement before adding complexity.

Vehicle service request tracking screenshot

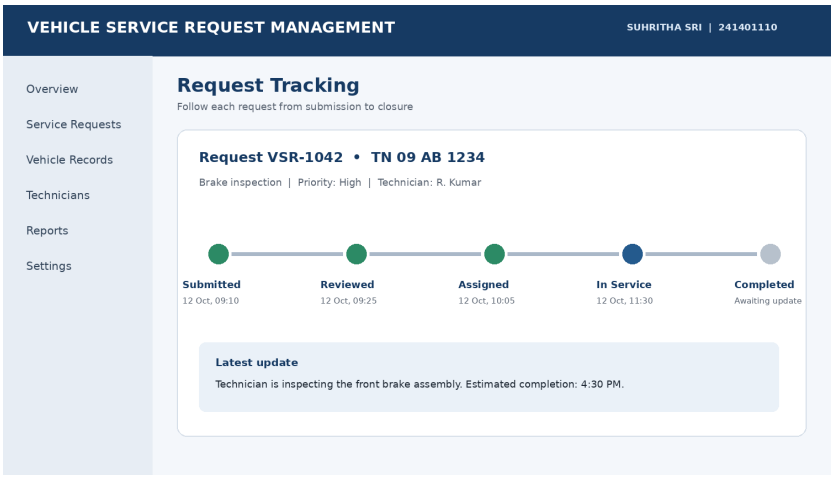


Fig. 4: Request tracking screen showing the service request lifecycle and status tracking.

3.8 Supporting interface screenshots

The following screens illustrate the vehicle service request workflow and its access-control layer. They are included as screenshots, not recreated as tables.

Service request form

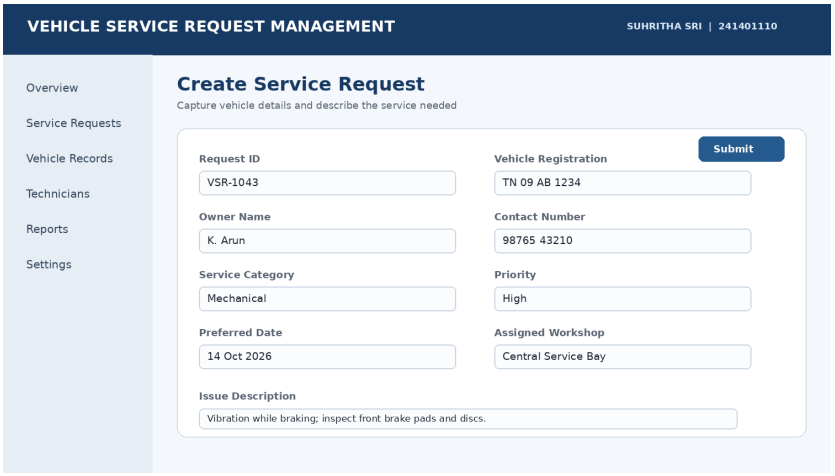


Fig. 5: Form used to enter customer, vehicle and requested service details.

Role-based access control

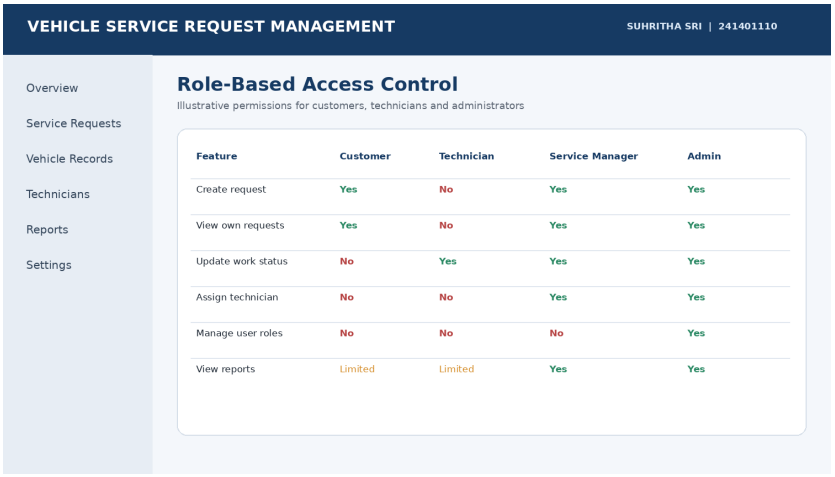


Fig. 6: Access-control screen illustrating role-based access to service request information.

3.9 Benefits and precautions

Aspect	Benefit	Precaution
Flexibility	A general-purpose language can implement specialised priority rules.	More code requires testing, review and maintenance.
Reuse	One Deluge function can be shared across multiple workflows.	Keep field names, inputs and response values consistent.
Separation	External processing keeps Creator workflows lighter.	Handle network errors, timeouts and unavailable endpoints.
Security	Managed connections can keep credentials out of source code.	Restrict access, validate inputs and avoid exposing secrets in logs.
Limits and cost	Serverless functions can scale with request volume.	Check current platform limits, execution quotas and pricing.
Service quality	Priority labels can help staff triage incoming requests.	Rules must be approved by the service centre and reviewed regularly.

4. Final Result

The experiment outlines a pro-code extension pattern for the Vehicle Service Request Management System using a reusable Deluge custom function in Zoho Creator and a serverless Zoho Catalyst function reached through a managed HTTPS connection. The worked example passes service type, urgency and breakdown status to a scoring function and uses its response to update the request priority. The report compares extension choices, presents illustrative code, documents the execution and failure paths, and records key security and maintenance precautions.

The objective of exploring pro-code extension was met at the design and study level. The endpoint and connection are placeholders, and the example was not deployed or tested; therefore, no live execution or operational result is claimed.

Reference note

Prepared as an illustrative lab exercise using the Zoho Creator / Deluge and Zoho Catalyst extension pattern. Before real deployment, verify the current platform documentation, supported connection configuration, API payload format, security settings and service-priority policy.